REPORT

Zajęcia: Analog and digital electronic circuits
Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 1
Date 27.02.26
Topic: "Spectral Analysis of Deterministic Signals"
Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Synthesize a discrete-time signal by using the IDFT in matrix notation for different values of N . Show the matrices W and K. Plot the signal synthesized.

2. Input data:
Variant 13:
xμ = [6, 2, 4, 3, 4, 5, 0, 0, 0, 0]T

3. Commands used (or GUI):

Digital Signal Processing – Laboratory

Spectral Analysis of Deterministic Signals

Task: Signal synthesis using the IDFT in matrix notation

In this notebook you will:

1. Define a spectrum vector (given in the exercise variants).
2. Build the **index matrix** and the **IDFT matrix** .
3. Synthesize the discrete-time signal using matrix multiplication for different values of .
4. Plot the synthesized signal (real/imaginary parts and magnitude).

Notation
– frequency index (DFT bin)
– time index
– IDFT length (number of samples in one period)

Source code:

```
import numpy as np
import matplotlib.pyplot as plt

# Make plots a bit nicer in Jupyter
plt.rcParams["figure.figsize"] = (9, 3.8)
plt.rcParams["axes.grid"] = True

np.set_printoptions(precision=3, suppress=True)
```

Theory: IDFT in matrix form

The IDFT is

[x[n] = \frac{1}{N} \sum_{\mu=0}^{N-1} X[\mu] , e^{j 2\pi \mu n/N}, \quad n=0,\dots,N-1.]

Define the **index matrix**

[K[n,\mu] = n\cdot \mu]

and the **IDFT matrix**

[W[n,\mu] = e^{j 2\pi K[n,\mu]/N}.]

Then the synthesis becomes a single matrix multiplication:

[\mathbf{x} = \frac{1}{N}, \mathbf{W}, \mathbf{X}.]

In this lab, the given vectors are interpreted as **DFT/IDFT coefficients** .

Choose the variant (given spectrum vector)

Each variant defines a column vector (real values in the assignment, but the IDFT result may be complex).

You can switch variants by changing `variant_id`. (Variant that given is **13**).

Source code:

```
# ----- Variant selection -----
variants = {
    1: np.array([6, 2, 4, 3, 4, 5, 0, 0, 0, 0], dtype=complex),
    2: np.array([10, 5, 6, 6, 2, 4, 3, 4, 5, 0, 0, 0, 0], dtype=complex),
    3: np.array([6, 2, 4, 3, 4, 5, 0, 0, 0, 0], dtype=complex),
    4: np.array([6, 2, 4, 3, 4, 5, 0, 0, 0], dtype=complex),
    5: np.array([6, 4, 4, 5, 3, 4, 5, 0, 0, 0, 0], dtype=complex),
}

variant_id = 1  # <-- change to 1..5

X = variants[variant_id].reshape(-1, 1)  # column vector (N0 x 1)
N0 = X.shape[0]

print(f"Variant {variant_id}: length N0 = {N0}")
print("X[μ] =")
print(X.T)
```

Output:

```
Variant 1: length N0 = 10
X[μ] =
[[6.+0.j 2.+0.j 4.+0.j 3.+0.j 4.+0.j 5.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]]
```

Helper functions: build *K*, build *W*, and perform the IDFT

We will build matrices explicitly to match the assignment statement.

Source code:

```
def build_K_W(N: int):
    """Return index matrix K and IDFT matrix W for a given length N."""
    n = np.arange(N).reshape(-1, 1)          # (N x 1)
    mu = np.arange(N).reshape(1, -1)        # (1 x N)
    K = n @ mu                               # (N x N) outer product => K[n,
μ] = n*μ
    W = np.exp(1j * 2 * np.pi * K / N)      # (N x N)
    return K, W

def idft_matrix(X: np.ndarray, N: int):
    """IDFT synthesis x = (1/N) * W * X, where X is (N x 1)."""
    K, W = build_K_W(N)
    x = (1 / N) * (W @ X)
    return x, K, W

def pad_or_trim_X(X: np.ndarray, N: int):
    """Return X resized to length N (zero-pad if needed, trim if too long)."""
    X = X.reshape(-1, 1)
    if X.shape[0] == N:
        return X
    if X.shape[0] < N:
        Xp = np.zeros((N, 1), dtype=complex)
        Xp[:X.shape[0], 0] = X[:, 0]
        return Xp
    # if longer, keep first N bins (simple trimming for the exercise)
    return X[:N, :]
```

Step 1: show matrices K and W for a *small* N

For large N the matrices are big, so we usually **display them for a small example** (e.g.,) and then use bigger for synthesis and plotting.

Source code:

```
N_show = min(8, N0)  # display up to 8x8 to keep it readable

X_show = pad_or_trim_X(X, N_show)
x_show, K_show, W_show = idft_matrix(X_show, N_show)

print(f"Showing matrices for N = {N_show}\n")

print("K matrix (n·μ):")
print(K_show)

print("\nW matrix = exp(j*2π*K/N):")
print(W_show)

print("\nSynthesized x[n] (for N_show):")
print(x_show.T)
```

Output:

```
Showing matrices for N = 8

K matrix (n·μ):
[[ 0  0  0  0  0  0  0  0]
 [ 0  1  2  3  4  5  6  7]
 [ 0  2  4  6  8 10 12 14]
 [ 0  3  6  9 12 15 18 21]
 [ 0  4  8 12 16 20 24 28]
 [ 0  5 10 15 20 25 30 35]
 [ 0  6 12 18 24 30 36 42]
 [ 0  7 14 21 28 35 42 49]]

W matrix = exp(j*2π*K/N):
[[ 1.   +0.j     1.   +0.j     1.   +0.j     1.   +0.j     1.   +0.j
   1.   +0.j     1.   +0.j     1.   +0.j   ]
 [ 1.   +0.j     0.707+0.707j  0.   +1.j    -0.707+0.707j -1.   +0.j
  -0.707-0.707j -0.   -1.j     0.707-0.707j]
 [ 1.   +0.j     0.   +1.j    -1.   +0.j    -0.   -1.j     1.   -0.j
   0.   +1.j    -1.   +0.j    -0.   -1.j   ]
 [ 1.   +0.j    -0.707+0.707j -0.   -1.j     0.707+0.707j -1.   +0.j
   0.707-0.707j  0.   +1.j    -0.707-0.707j]
 [ 1.   +0.j    -1.   +0.j     1.   -0.j    -1.   +0.j     1.   -0.j
  -1.   +0.j     1.   -0.j    -1.   +0.j   ]
 [ 1.   +0.j    -0.707-0.707j  0.   +1.j     0.707-0.707j -1.   +0.j
   0.707+0.707j -0.   -1.j    -0.707+0.707j]
 [ 1.   +0.j    -0.   -1.j    -1.   +0.j     0.   +1.j     1.   -0.j
  -0.   -1.j    -1.   +0.j    -0.   +1.j   ]
 [ 1.   +0.j     0.707-0.707j -0.   -1.j    -0.707-0.707j -1.   +0.j
  -0.707+0.707j -0.   +1.j     0.707+0.707j]]
```

```
Synthesized x[n] (for N_show):
[[ 3.  +0.j  -0.28+0.5j  0.75+0.5j  0.78-0.5j  0.5 +0.j   0.78+0.5j
   0.75-0.5j -0.28-0.5j]]
```

Step 2: synthesize and plot for different values of N

Idea: Changing N changes the assumed period and the frequency grid.
For this lab, we will keep the given coefficients and:

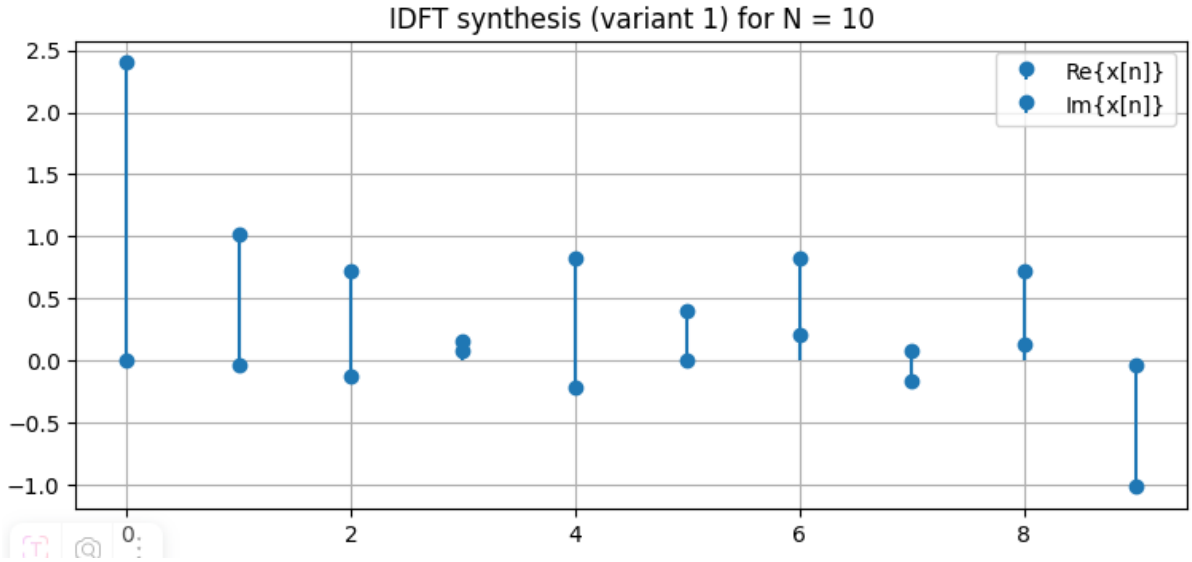
- Use $N = N0$ (natural length of the provided vector),
- And a larger N (zero-padding the spectrum), e.g. $N = 2N0$.

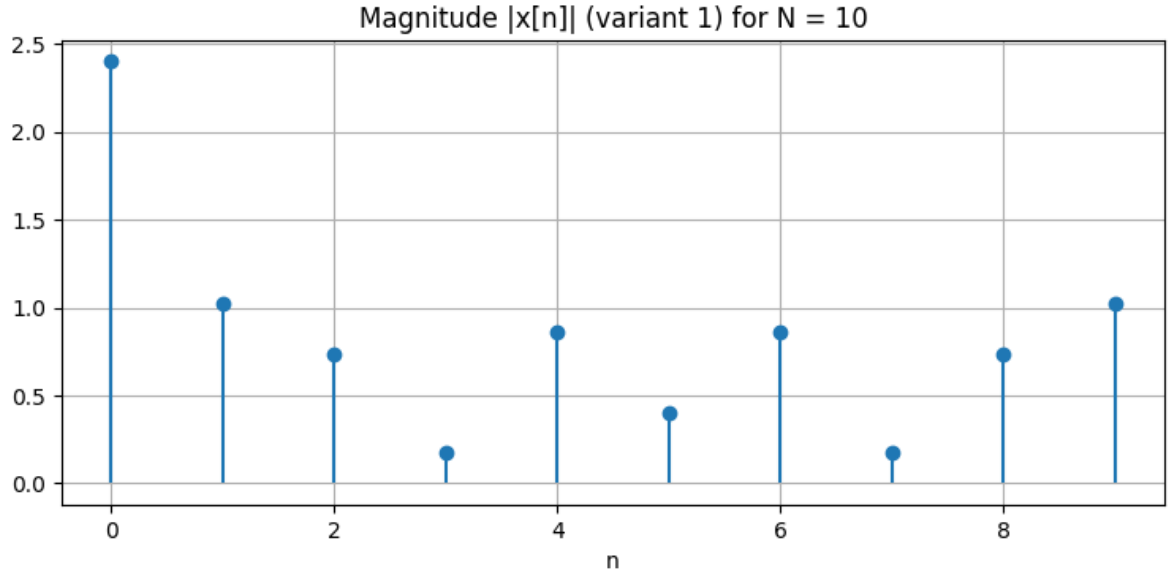
You can change `N_list` to any values you want (e.g., 16, 32, ...).

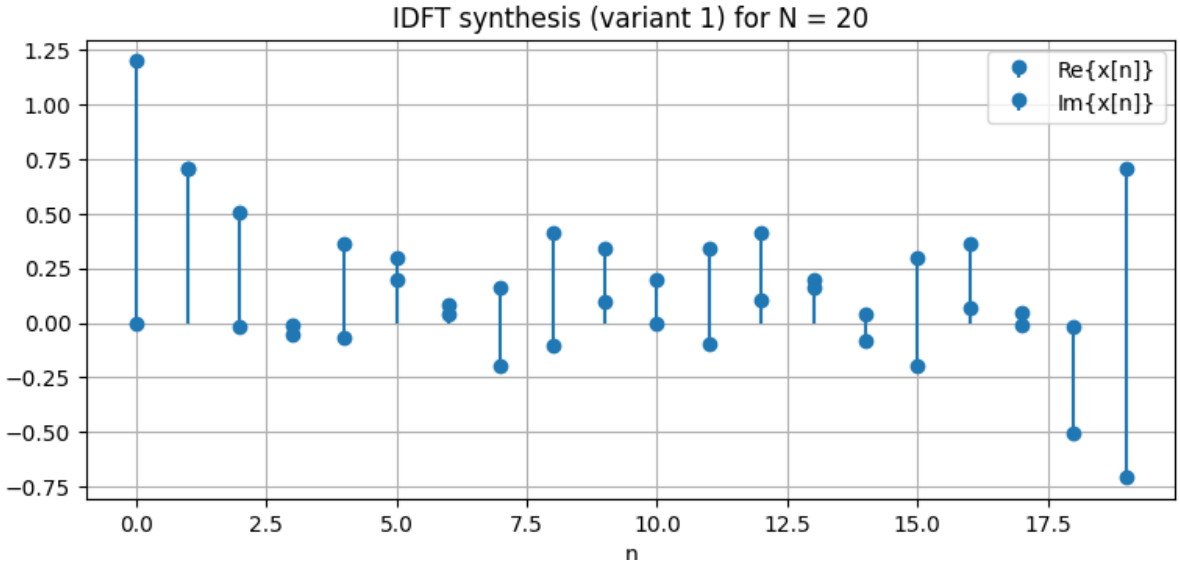
Source code:

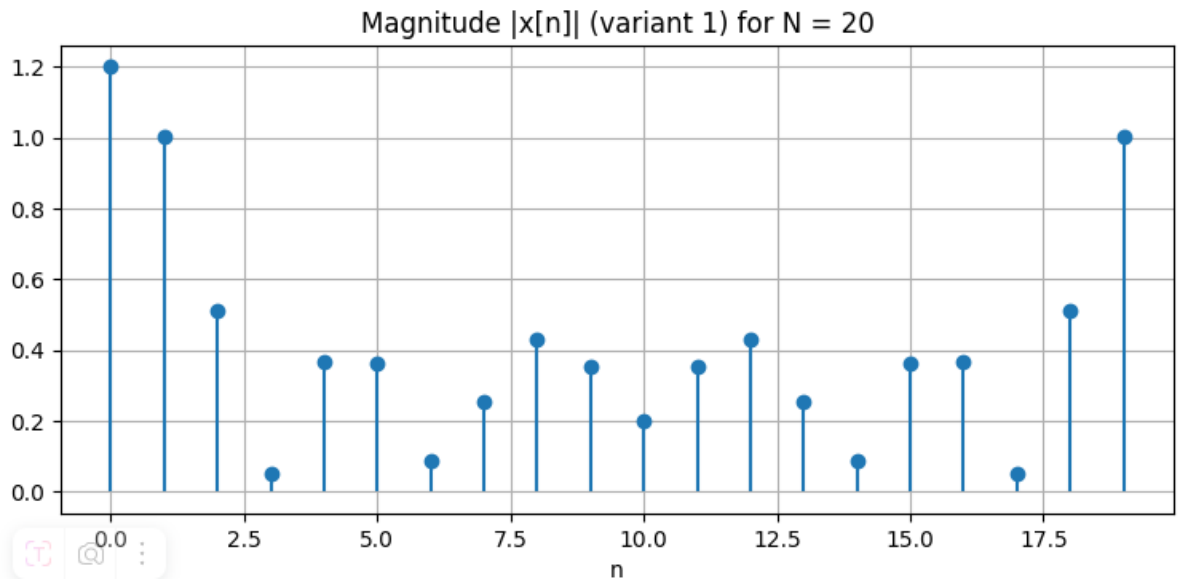
```python
import numpy as np
import matplotlib.pyplot as plt

N_list = [N0, 2*N0]  # try also [N0, 16, 32] if you want

for N in N_list:
    XN = pad_or_trim_X(X, N)
    xN, K, W = idft_matrix(XN, N)

    n = np.arange(N)

    plt.figure()
    # Real and imaginary parts
    plt.stem(n, np.real(xN[:, 0]), basefmt=" ", label="Re{x[n]}")
    plt.stem(n, np.imag(xN[:, 0]), basefmt=" ", label="Im{x[n]}")
    plt.title(f"IDFT synthesis (variant {variant_id}) for N = {N}")
    plt.xlabel("n")
    plt.legend()
    plt.show()

    plt.figure()
    # Magnitude
    plt.stem(n, np.abs(xN[:, 0]), basefmt=" ")
    plt.title(f"Magnitude |x[n]| (variant {variant_id}) for N = {N}")
    plt.xlabel("n")
    plt.show()
```

Output:

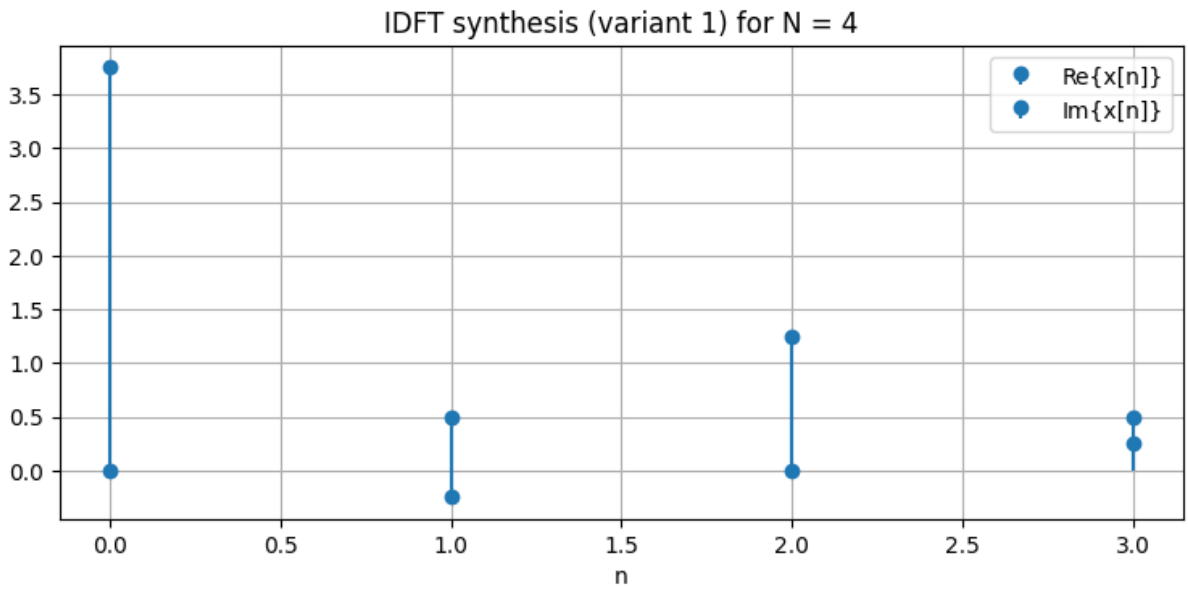

IDFT synthesis (variant 1) for N = 10
Re{x[n]}
Im{x[n]}
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
0
2
4
6
8

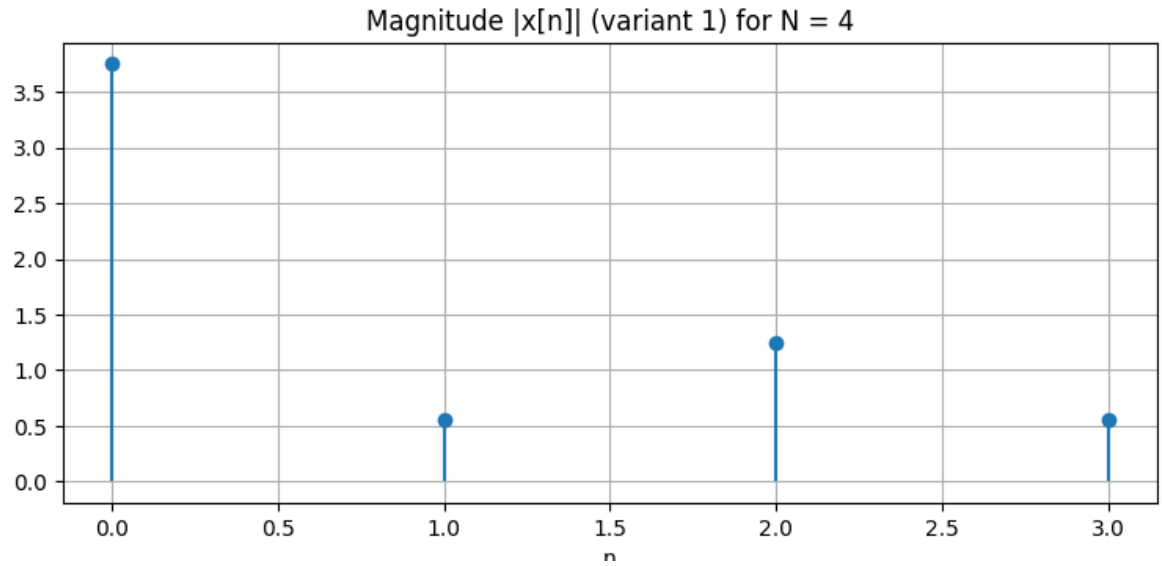

Magnitude |x[n]| (variant 1) for N = 10
2.5
2.0
1.5
1.0
0.5
0.0
0
2
4
6
8
n

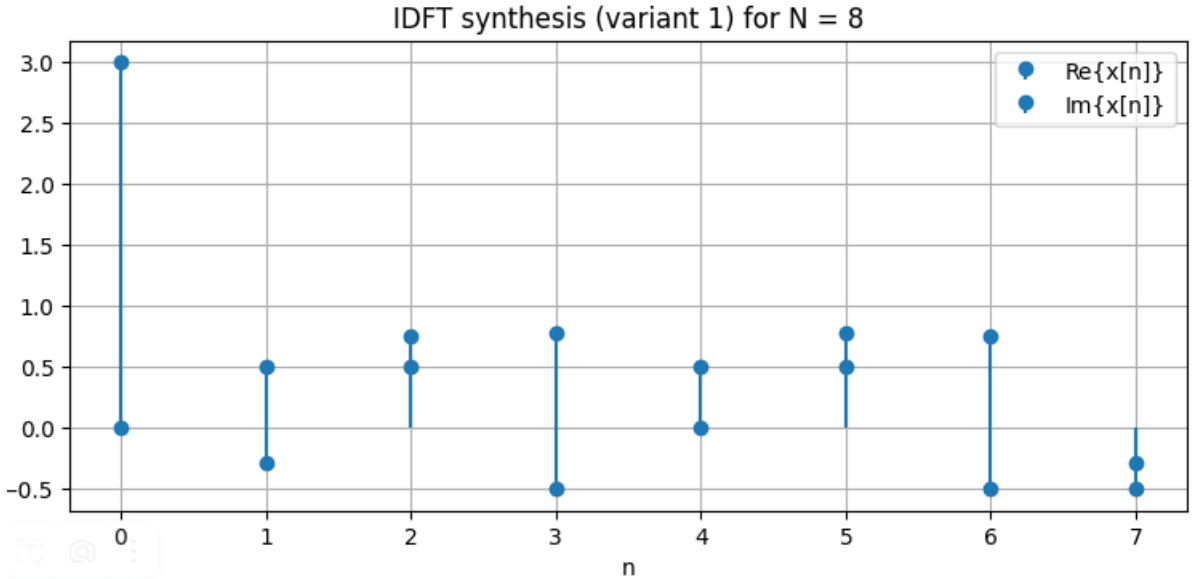

IDFT synthesis (variant 1) for N = 20
Re{x[n]}
Im{x[n]}
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
n

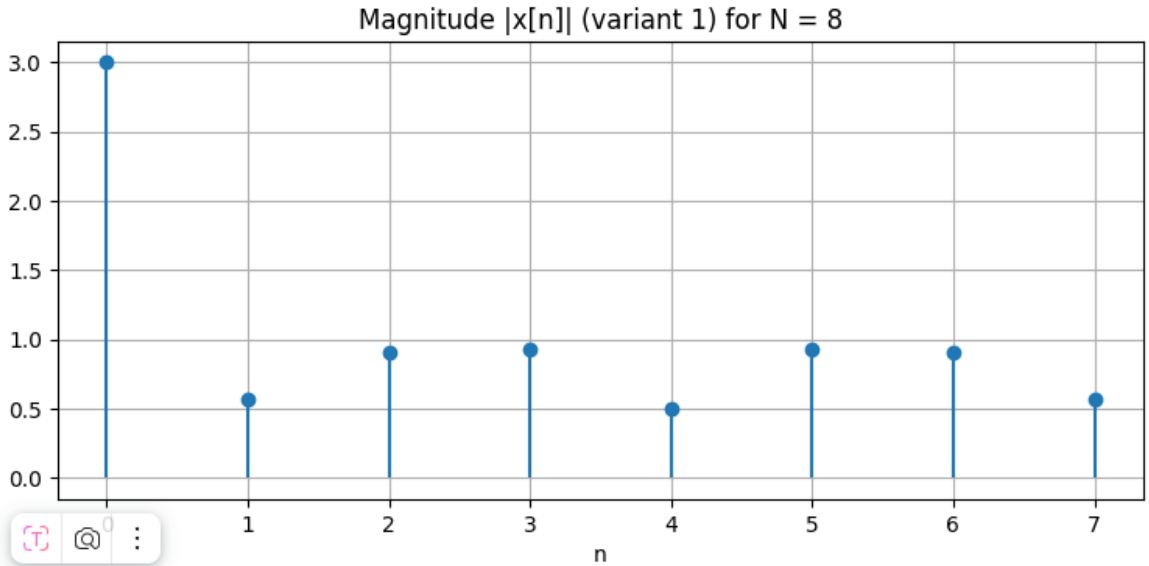


Diagramms for N_list = [4, 8]




Magnitude |x[n]| (variant 1) for N = 4
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
n


IDFT synthesis (variant 1) for N = 8
Re{x[n]}
Im{x[n]}
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
0
1
2
3
4
5
6
7
n


Magnitude |x[n]| (variant 1) for N = 8
3.0
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6
7
n

Question:

Is x**[*n*]** real or complex? how does increasing affect the shape?

The synthesized signal x[n] is generally complex, because it has both real and imaginary parts. The real part corresponds to the in-phase component, and the imaginary part to the quadrature component. Increasing N (the number of points in the IDFT) increases the resolution of the signal. With larger N, the waveform becomes smoother and more detailed, and its discrete representation better approximates the continuous-time signal.

Link to remote repozytorium (e.g. GitHub)

Conclusion:

The discrete-time signal x[n] synthesized using the IDFT is generally complex, containing both real and imaginary parts. Increasing the IDFT length N improves the resolution of the signal: the waveform becomes smoother, more detailed, and better represents the underlying continuous-time signal. The matrices K and W clearly show how the time and frequency indices interact in the synthesis process. This demonstrates that IDFT in matrix form is a practical and flexible method for signal reconstruction and for analyzing the effects of spectrum zero-padding.